Harness SDE-1 Online Assessment

Coding questions · October 2026

Problem 1: Minimum Jumps in a Grid

You are given an `n × m` grid where each cell is either `0` (empty) or `1` (obstacle), and an integer `k`.

You start at cell `(0, 0)` and want to reach cell `(n − 1, m − 1)`.

In one move, you can jump `x` cells in any of the 4 directions, where `1 ≤ x ≤ k`. From cell `(i, j)`:

- **Right:** move to `(i, j + x)`
- **Left:** move to `(i, j − x)`
- **Down:** move to `(i + x, j)`
- **Up:** move to `(i − x, j)`

A jump is valid only if:

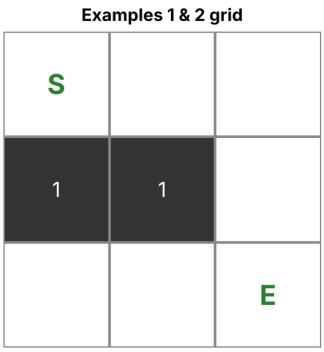
- the destination stays inside the grid, and
- **every cell along the jump**, including the destination, is empty (you cannot jump over or land on an obstacle).

Each jump costs **1**, regardless of its length.

Return the **minimum number of jumps** to reach `(n − 1, m − 1)`. If it is impossible, return `−1`.

Example 1

```
grid = [[0,0,0],
        [1,1,0],
        [0,0,0]]
k = 2
Output: 2
```



Explanation: Jump right by 2 to `(0,2)`, then down by 2 to `(2,2)`. S marks the start, E the end, and dark cells are obstacles.

Example 2

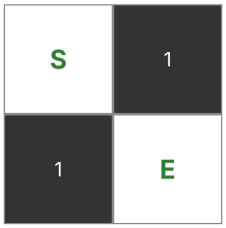
```
Same grid as Example 1, k = 1
Output: 4
```

Explanation: `(0,0) → (0,1) → (0,2) → (1,2) → (2,2)`, one cell at a time.

Example 3

```
grid = [[0,1],
        [1,0]]
k = 1
Output: -1
```

Example 3 grid



Explanation: Both moves out of `(0,0)` are blocked, so the end cannot be reached.

Constraints

- `1 ≤ n, m ≤ 100`
- `1 ≤ k ≤ 100`
- `grid[i][j]` is `0` or `1`

Problem 2: Tower Placement on a Tree

You are given a tree with `n` nodes numbered `0` to `n − 1`, described by two arrays `treeNode_from` and `treeNode_to` of length `n − 1`, where each pair forms an undirected edge.

You may place a tower on any **subset** of nodes (including none or all), so there are 2^n possible placements.

A node is **active** if:

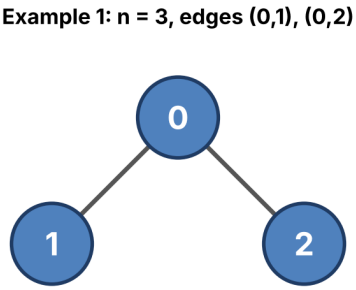
- it has a tower on it, or
- it is directly connected to a node that has a tower.

For every `c` from `0` to `n`, count the number of placements that make **exactly** `c` **nodes active**.

Return an array of length `n + 1`, where index `c` holds that count.

Example 1

```
n = 3
treeNode_from = [0, 0]
treeNode_to   = [1, 2]
Output: [1, 0, 2, 5]
```



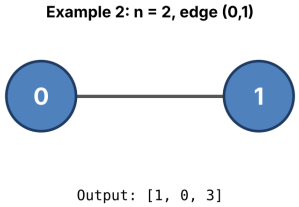
Explanation: node 0 is connected to nodes 1 and 2.

Towers on	Active nodes	Count
none	—	0
{0}	{0, 1, 2}	3
{1}	{0, 1}	2
{2}	{0, 2}	2
{0,1}, {0,2}, {1,2}, {0,1,2}	{0, 1, 2}	3 (each)

So 1 placement gives 0 active nodes, no placement gives exactly 1, 2 placements give 2, and 5 placements give 3.

Example 2

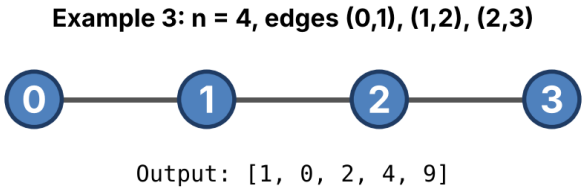
```
n = 2
treeNode_from = [0]
treeNode_to   = [1]
Output: [1, 0, 3]
```



Explanation: with no towers, nothing is active. Any non-empty placement ({0}, {1} or {0,1}) activates both nodes.

Example 3

```
n = 4
treeNode_from = [0, 1, 2]
treeNode_to   = [1, 2, 3]
Output: [1, 0, 2, 4, 9]
```



Explanation (path 0–1–2–3): a tower only on node 0 or only on node 3 activates 2 nodes; {1}, {2}, {0,1} and {2,3} each activate 3; the remaining 9 placements activate all 4. Total $1 + 0 + 2 + 4 + 9 = 16 = 2^4$.

Constraints

Not recorded. Fill in from the original problem (likely bounds on n, and possibly a modulo such as $10^9 + 7$, since counts grow like 2^n).